AI辅助Git操作教学案例

Git概念快速入门（边做边学）

> 本章采用"概念 + 实践"的方式，让你在动手操作中理解Git核心概念。每个概念都配有可以直接对AI说的提示词。

---

1. 什么是 Commit（提交）？

概念：给项目拍一张快照

想象你在画一幅画，每隔一段时间你会用手机拍一张照片记录进度。Git的Commit就是这样的"照片"——它记录了某一时刻项目所有文件的完整状态。

每个Commit包含：

- 谁在什么时候做了修改
- 修改了哪些文件
- 这次修改的描述信息（Commit Message）

实践：创建你的第一个Commit

1. 打开一个项目文件夹
2. 对AI说：

```
帮我初始化Git仓库，然后提交当前所有文件
```

3. AI会自动：
    - 执行 `git init` 初始化仓库
    - 添加所有文件到暂存区
    - 生成规范的提交信息并完成提交

验证：

```
查看最近的提交历史
```

---

2. 什么是 Branch（分支）？

概念：平行的时间线

想象你在玩一个角色扮演游戏，在关键剧情点前你存了一个档。然后你沿着不同的选择玩了下去——这就是分支。

- `main`（或`master`）分支：游戏的主线剧情，稳定可用的版本
- `feature/xxx` 分支：你在开发新功能时的存档

为什么用分支?

- 开发新功能时不影响主线
- 多人协作时各自在独立分支工作
- 可以随时切换回稳定版本

功能分支工作流（Feature Branch Workflow）

功能分支工作流是最常用的Git协作模式，核心理念是：**所有功能开发都在独立分支完成，通过合并集成到主线。**

工作流程图

```
main分支（稳定）
    │
    ├─ 创建 feature/inventory 分支
    │       │
    │       ├─ 开发背包系统
    │       ├─ Commit 提交
    │       │
    │   合并回main
    │       │
    ▼       ▼
main分支（包含背包系统）
```

完整操作流程

步骤1：从main创建功能分支

```
帮我从main分支创建一个 feature/inventory 分支，我要开发背包系统
```

AI可能依次完成：

- 切换到main分支
- 拉取最新代码
- 创建并切换到新分支

步骤2：在功能分支上开发

正常编写代码，然后提交：

```
帮我提交这次的git修改，这次实现了背包界面和物品拖拽功能
```

步骤3：推送到远程仓库

```
帮我把 feature/inventory 分支推送到GitHub
```

AI可能依次完成：

- 推送分支到远程
- 设置上游分支

步骤4：合并到main分支

开发完成并通过测试后，合并回主线：

```
帮我把 feature/inventory 分支合并到 main 分支
```

AI可能依次完成：

- 切换到main分支
- 拉取最新代码
- 合并功能分支
- 处理可能的冲突
- 推送合并后的main到远程

步骤5：删除功能分支

```
帮我删除 feature/inventory 分支（本地和远程）
```

常见场景速查

场景	AI提示词
初始化项目	帮我初始化Git仓库，创建.gitignore，提交所有文件
查看状态	查看当前Git状态，告诉我修改了哪些文件
创建功能分支	从main创建 feature/xxx 分支
提交代码	帮我提交修改，描述：xxx
推送到远程	帮我把当前分支推送到GitHub
合并分支	把 feature/xxx 合并到main
解决冲突	合并出现冲突，帮我解决
切换分支	切换到 xxx 分支

核心观点

传统Git学习曲线陡峭，但有了AI工具（如Trae、Claude Code、Cursor等），只需自然语言即可完成复杂操作。

对比项	传统方式	AI辅助方式
学习成本	需记忆50+命令	自然语言描述意图
错误率	高（命令参数复杂）	低（AI生成正确命令）
冲突处理	手动解决，易出错	AI智能协助
Commit Message	手写，不规范	AI自动生成规范格式
入门时间	数周到数月	几乎即时

总结

功能分支工作流是团队协作的基础：

1. **main分支始终保持稳定可运行**
2. **每个功能在独立分支开发**
3. **开发完成后合并回main**
4. **合并后删除功能分支**

使用AI辅助，你只需用自然语言描述意图，无需记忆复杂命令。这让Git从"需要学习的技能"变成了"可以直接使用的工具"。